

Common Instructions: For this assignment you will need to write a program to solve a problem, based on handout code provided on D2L. You should upload your solution to D2L in individual plaintext files with the correct extension for source code (i.e. `.py` for Python, `.c` or `.h` for C). In general, you do not need to submit handout code files that are unchanged; you only need to upload code files that you modified in some way.

Project Description: The `paging_simulation` program, which is available on D2L in the compressed folder `paging-simulation.zip`, simulates a memory-management system for a single process that can be configured with several different page replacement algorithms. The simulator loads a sequence of virtual memory accesses from a data file, constructs a “page table manager” object that uses the requested algorithm, and then asks the page table manager to translate each memory address in the sequence. The page table manager counts the number of page faults that occur (assuming it is configured with a physical memory size smaller than the virtual memory size), and the simulator prints out this total at the end.

This program uses a form of polymorphism in C to represent the page table managers: The simulator’s main loop interacts with a `pager_interface`, which is a `struct` that contains several function pointers representing interface methods. When the program starts up, it creates the `pager_interface` by calling one of the 5 different “`pager_new`” functions (`fifo_pager_new`, `lru_pager_new`, `clock_pager_new`, `random_pager_new`, or `opt_pager_new`), each of which binds a different set of “method” functions to the `pager_interface`’s function pointers, based on which algorithm was selected in the command-line arguments. The “method” functions, like other object-oriented C functions, always take as their first argument a pointer to the `pager_interface` object that called them, which can be used like the `this` pointer or `self` reference in other languages. The Random page table manager has already been implemented, and you can use it to test the simulation before you write any of your own code.

Part 1: Complete the implementations of the LRU, FIFO, Clock, and OPT page replacement strategies by adding code to `lru_pager.c`, `fifo_pager.c`, `clock_pager.c`, and `opt_pager.c`. You *must* provide an implementation of the `handle_page_fault` method for each pager, and you *may* optionally provide a non-default implementation of the `translate` method to add additional book-keeping for the page replacement algorithm each time a page is accessed. If you do write your own `translate` method for a pager, be sure to replicate the functionality of the `default_translate` method (either by calling that function or by writing similar code) so that address translation still works as expected and the `page_faults` counter is still incremented. To help you see how to use polymorphism in C, `dummy_pager.c` contains a simple implementation of the pager interface that increments a counter each time `translate` is called, and implements `handle_page_fault` by either returning an unused frame or terminating the program if memory is full (it does not evict anything).

Part 2: The files in the `data` folder, included with the paging-simulation project, contain traces of array accesses of various sorting algorithms when run on an input array of size 1024 (the insertion sort and bogosort traces have been truncated to 300,000 accesses). Compare the number of page faults between the different algorithms when run on these traces for a variety of memory sizes (i.e.

change the page size and the number of frames in the program's command-line arguments). Do different algorithms perform better for different traces or different memory sizes? Explain your observations. **Note:** Submit your response to this part in a PDF file, like other written responses.